David Mohaisen, Professor
University of Central Florida
Department of Computer Science
Cybersecurity and Privacy Cluster

https://cs.ucf.edu/~mohaisen/

Decoding Strategies for Language Models

① Recap
② Introduction to decoding
③ Impact of decoding
④ Sampling and temperature
⑤ Comparison
⑥ Conclusion and takeaways

Objectives

- Review **general vs. conditional language models**.
- Understand **teacher forcing** in training RNN/Seq2Seq models.
- Compare decoding strategies: **greedy, beam search, sampling**.
- Analyze the impact of **beam size and diversity trade-offs**.
- Explain the role of **top-k sampling and softmax temperature**.

Recap

Recap of Language Models (LMs)

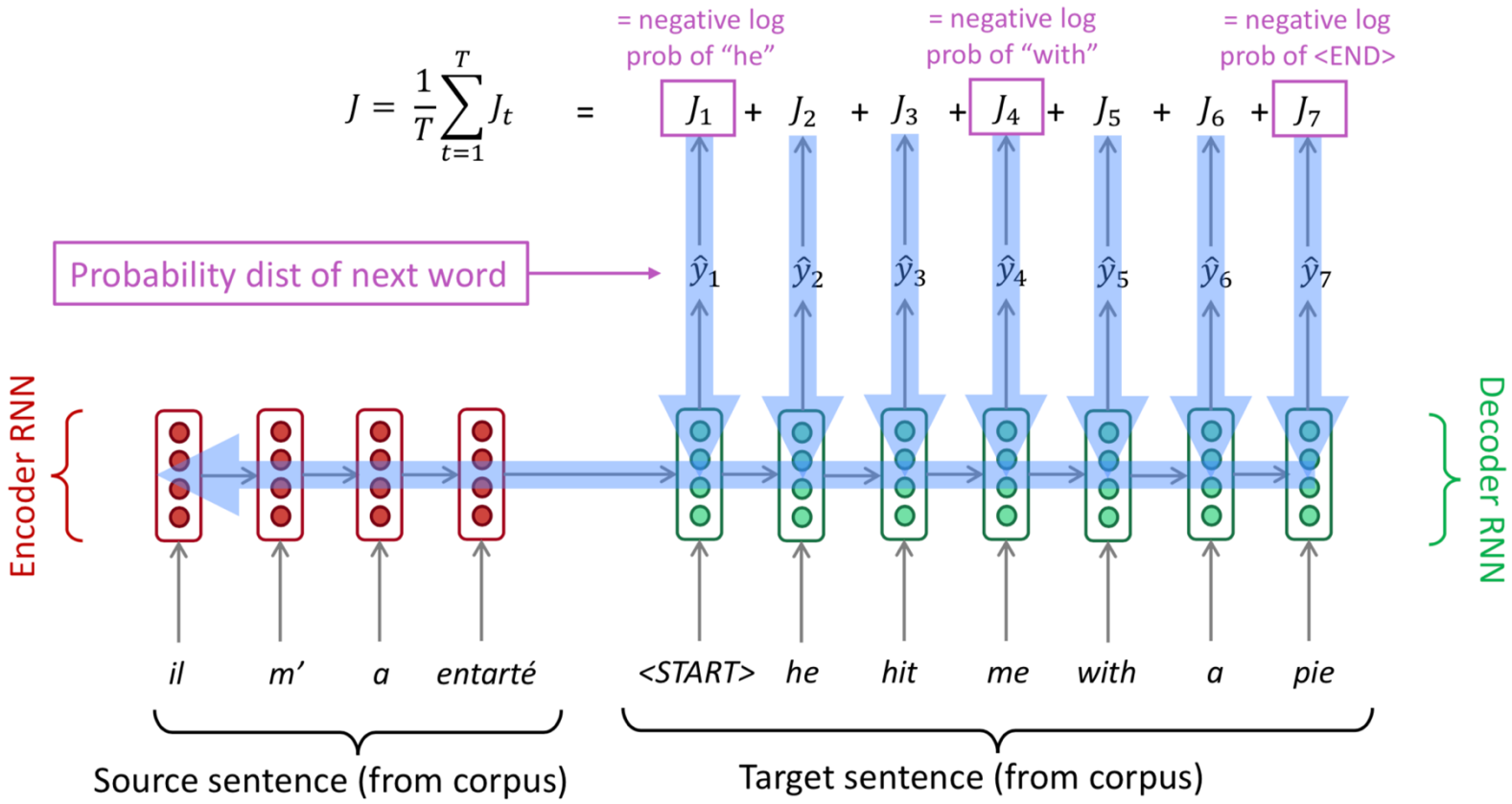
- **Types of Language Models**
  - **General Language Models**: Predict text fluency and coherence.
  - **Conditional Language Models**: Guide text generation based on input.
  - **RNN-LMs**: Recurrent models that generate sequences based on learned probabilities.

- **Conditional Language Models:** Predict the next word given previous words and additional input x; $P(y_t | y_1 \ldots y_{t-1}, x)$
  - **Summarization**: x = input text, y = summary.
  - **Translation**: x = source language, y = translated text.
  - **Dialogue**: x = conversation history, y = next utterance.

General LMs for Text Fluency and Coherence

- **What Are General Language Models?**
  - Models trained to predict the **next word** in a sequence based on prior context.
  - Learn **grammar, semantics, and discourse coherence** from large text corpora.
- **Key Objectives are Fluency and Coherence**
  - **Fluency**: Ensure generated text is **grammatically correct and natural-sounding**.
  - **Coherence**: Maintain **logical flow** across sentences and paragraphs.
- **Examples of General LMs**
  - **n-gram Models**: Basic statistical approach for next-word prediction.
  - **RNN-based LMs**: Use memory to capture **short-term dependencies**.
  - **Transformer-based LMs**: GPT (autoregressive models for fluency) and BERT/T5 (context-aware for coherence)
- **How They Improve Fluency & Coherence**
  - **Token Probability Prediction**: Assigns high probabilities to natural word sequences.
  - **Contextual Embeddings**: Captures deep meaning relationships.
  - **Long-Range Dependencies**: Avoids incoherence by considering broader context.

Training (Conditional) RNN-LM



During training, we feed the gold (aka reference) target sentence into the decoder, regardless of what the decoder predicts. This training method is called "**Teacher Forcing**"

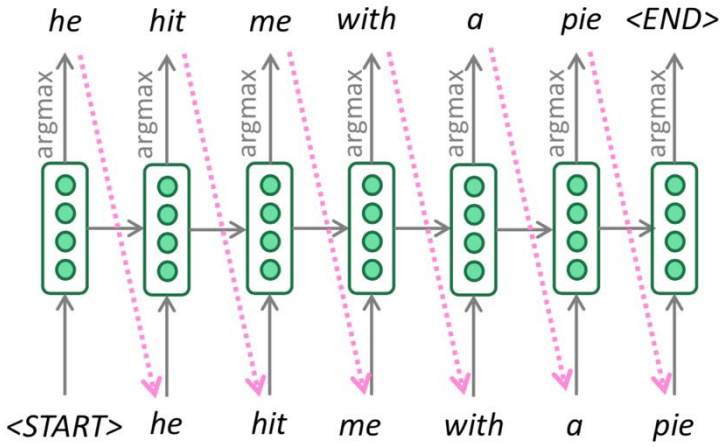
Introduction to Decoding

Conditional LMs and Decoding

- After training a **conditional language model**, a **decoding algorithm** is used to generate text.
- Two main decoding approaches:
- **Greedy Decoding**
  - At each step, selects the **word with the highest probability**.
  - Fast but can lead to **suboptimal sequences** (locally optimal but not globally best).
- **Beam Search**
  - Expands multiple possible sequences (**beam width > 1**).
  - Selects the **most probable sequence overall**, improving coherence.
  - Slower than greedy decoding but produces **higher-quality text**.

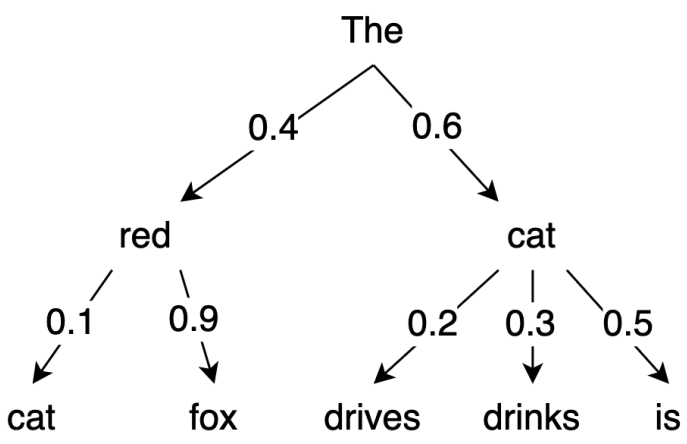
Greedy Decoding

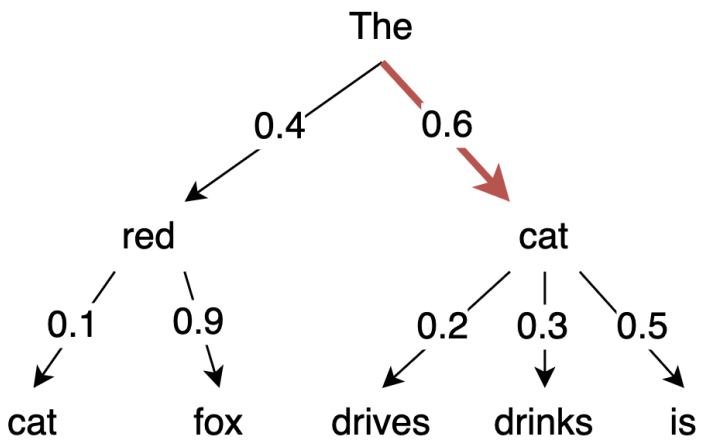
- **Steps of Greedy Decoding**
  - Start with an initial input (e.g., prompt or context).
  - Predict the **next word** by selecting the **highest probability** token.
  - Use that word as input for the next prediction.
  - Repeat until the **end of the sequence**.



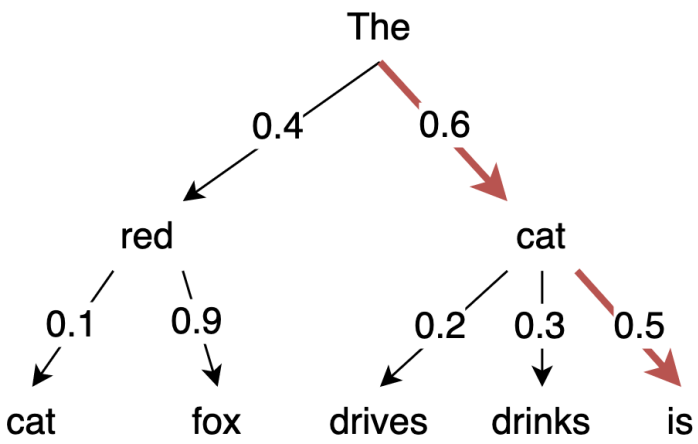
- **Problems with Greedy Decoding**
  - **Lack of Bootstrapping**: Doesn't explore alternative
    monotonic outputs.
  - **Poor Output Quality**: Can result in **unnatural or g**
    sentences.
  - **Suboptimal Global Coherence**: Ignores long-term dependencies, making the generated text less structured.

Greedy Decoding



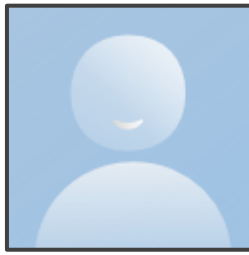


Max(0.4, 0.6)? 0.6 → The cat



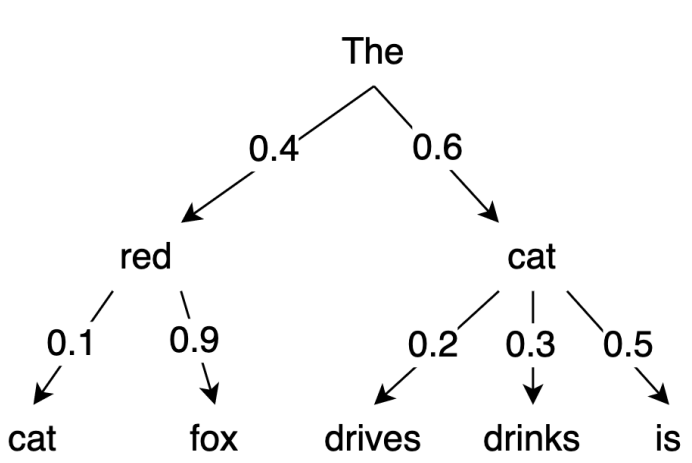
Max(0.2, 0.3, 0.5)? 0.5 → cat is

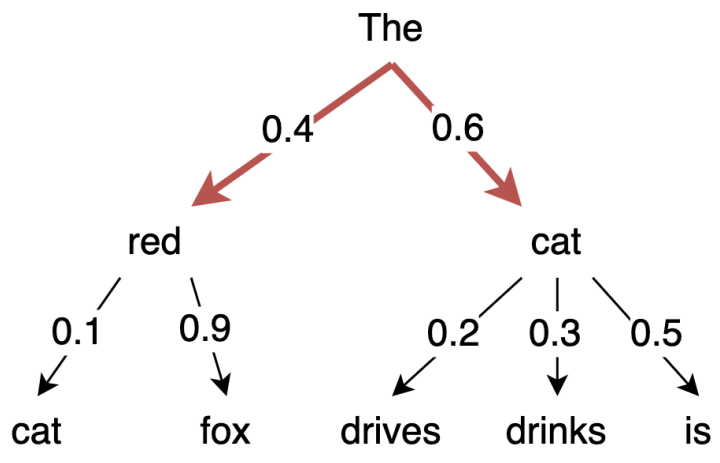
Beam Search Decoding

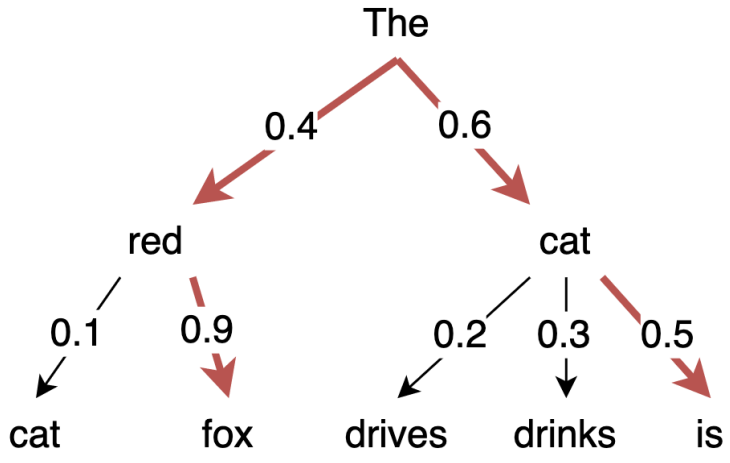


- **How It Works**
  1. At each decoding step, keep the **k most probable partial sequences** (**beam size = k**).
  2. Expand each hypothesis by predicting the next possible words.
  3. Continue until a **stopping criterion** is met (e.g., reaching an end token).
  4. Select the **highest-probability sequence**, adjusting for **length normalization** to avoid favoring shorter sentences.
- **Advantages**
  - **More flexible** than greedy decoding.
  - **Improves coherence and fluency** in generated text.

Beam Search Decoding







🧐 The red 0.4
🧐 The cat 0.6

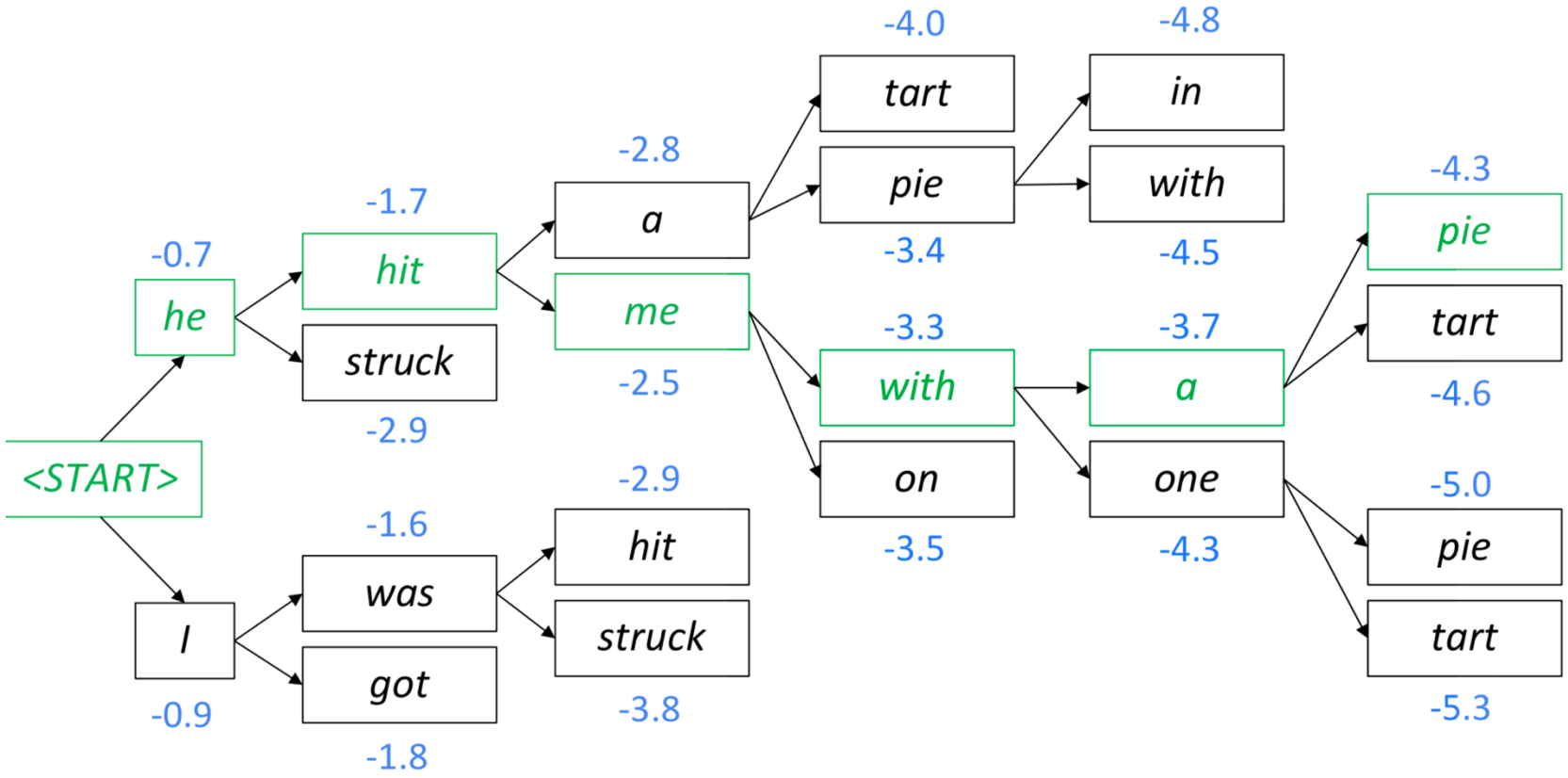
🧐 The read cat = 0.4×0.1 = 0.04
🧐 The red fox = 0.4×0.9 = 0.36
🧐 The cat drives = 0.6×0.2 = 0.12
🧐 The cat drinks = 0.6×0.3 = 0.18
🧐 The cat is = 0.6×0.5 = 0.30

🧐 The red fox = 0.4×0.9 = 0.36
🧐 The cat is = 0.6×0.5 = 0.30

✅ The red fox = 0.4×0.9 = 0.36
❌ The cat is = 0.6×0.5 = 0.30

Beam Search Decoding

Beam size = k = 2. Blue numbers = $\text{score}(y_1, \ldots, y_t) = \sum_{i=1}^{t} \log P_{\text{LM}}(y_i | y_1, \ldots, y_{i-1}, x)$



Impact of Decoding

Impact of Beam Size *k*

- **Effect of Small k (e.g., *k*=1) - s**imilar to **greedy decoding**, leading to:
  - **Ungrammatical and unnatural outputs**.
  - **Nonsensical or incorrect sentences**.
- **Effect of Larger *k:***
  - Considers **more hypotheses**, reducing errors.
  - Improves **fluency and coherence** in generated text.
  - Comes at a **higher computational cost**.
- **Problems with Very Large *k***
  - **Neural Machine Translation (NMT)**:
    - Increasing *k* too much **lowers BLEU score** due to **shorter translations**.
  - **Open-ended tasks (e.g., chit-chat bots)**:
    - **High k makes responses too generic**, reducing diversity.

Chitchat Bot & Impact of k

I mostly eat a lot of fresh and raw diet, so I save on groceries

Human chit-chat partner

Beam size	Model response
1	*I love to eat healthy and eat healthy*
2	*That is a good thing to have*
3	*I am a nurse so I do not eat raw food*
4	*I am a nurse so I am a nurse*
5	*Do you have any hobbies?*
6	*What do you do for a living?*
7	*What do you do for a living?*
8	*What do you do for a living?*

Low beam size: More on-topic but nonsensical; bad English

High beam size: Converges to safe, "correct" response, but it's generic and less relevant

Sampling and Temprature

Sampling-based Decoding

- **Pure Sampling:** At each step t, **randomly sample** from the probability distribution over possible next words.
  - Similar to **greedy decoding**, but instead of choosing the highest-probability word (**argmax**), it picks a word **randomly** based on its probability.

- **Top-n Sampling:** Instead of sampling from the **entire vocabulary** (V), restrict to the **n most probable words** at each step.
  - A trade-off between **pure sampling** and **greedy decoding**.
  - n=1 → **Greedy decoding** (always chooses the highest probability word).
  - n=V → **Pure sampling** (samples from the full probability distribution).
  - **Increasing n** → More **diverse, creative, and risky** output.
  - **Decreasing n** → More **generic and safe** output.

Softmax Temperature in Decoding

- **What is Softmax Temperature?**
  - A **scaling factor** applied to the prob. distribution before sampling the next word.
  - Controls the **level of randomness** in text generation. Formula:

$$P(y_t) = \frac{\exp(\frac{z_t}{T})}{\sum_i \exp(\frac{z_i}{T})}$$

- Where:
  - z_t = raw logits (unscaled probabilities).
  - T = temperature parameter.

Low T (<1.0): Makes the distribution **sharper** (high-prob. words dominate).
More deterministic, repetitive, and safe outputs.
Similar to **greedy decoding** at very low T.

High T (>1.0): Makes the distribution **smoother** (more diversity word choice).
More creative, diverse, and risky text generation.
Can lead to **incoherent or nonsensical** outputs if too high.

Choosing the Right T:

1 For factual tasks (e.g., translation, summarization): Use lower T (e.g., 0.7-1.0).

2 For creative writing (e.g., poetry, storytelling): Use higher T (e.g., 1.2-1.5).

Decoding Algorithms: Summary

- **Greedy decoding** is a simple method; gives low quality output
- **Beam search** (especially with high beam size) searches for high-probability output
  - Delivers better quality than greedy, but if beam size is too high, can return high-probability but unsuitable output (e.g. generic, short)
- **Sampling methods** are a way to get more diversity and randomness
  - Good for open-ended / creative generation (poetry, stories)
  - Top-n sampling allows you to control diversity
- **Softmax temperature** is another way to control diversity
  - It’s not a decoding algorithm! It's a technique that can be applied alongside any decoding algorithm.

Comparison and Takeaways

Decoding Algorithms: Summary

Decoding Method	Description	Pros	Cons
Greedy Decoding	Selects the most probable word at each step (**argmax**).	Simple, fast	Low-quality, repetitive, lacks diversity
Beam Search	Keeps track of the top **k** most probable sequences, selecting the highest probability sequence at the end.	Higher-quality than greedy, balances probability	Large **k** can lead to generic, short, high-probability but unsuitable text
Sampling-Based Decoding	Randomly samples from the probability distribution at each step.	Increases diversity, good for open-ended tasks (stories, poetry)	Can be incoherent if randomness is too high
Top-n Sampling	Samples from only the **top-n** most probable words instead of the entire vocabulary.	Balances diversity and coherence	Small **n** makes output generic; large **n** increases randomness
Softmax Temperature *(Not a decoding algorithm!)*	Adjusts probability distribution before sampling (low **T** = safe output, high **T** = diverse output).	Can be applied with any decoding method, controls creativity	Too high **T** reduces coherence, too low **T** reduces diversity

Takeaways

- Training uses teacher forcing; inference requires **decoding algorithms**.
- **Greedy decoding** is fast but often suboptimal.
- **Beam search** improves probability but may produce generic outputs.
- **Sampling methods** increase diversity for open-ended tasks.
- **Temperature controls randomness** and can be combined with any decoding method.